them texts to process. Eventually, they respond, and the count is added to the master state. The following listing shows the output logging two types of variables from the master when running `CounterApp`: the lag and the current word count.

Listing 8.6 Aggregator output from the master actor with aggregator nodes

```
[17:50:14,499] [DEBUG]--receiving, current lag 0                          ❶
[17:50:14,505] [DEBUG]--current count HashMap(this -> 16, a -> 32, very ->
➥16, simulates -> 16, simple -> 16, stream -> 32)                        ❷
[17:50:14,505] [DEBUG]--current count HashMap(this -> 17, a -> 34, very ->
➥17, simulates -> 17, simple -> 17, stream -> 34)
[17:50:14,505] [DEBUG]--current count HashMap(this -> 18, a -> 36, very ->
➥18, simulates -> 18, simple -> 18, stream -> 36)
[17:50:14,505] [DEBUG]--current count HashMap(this -> 19, a -> 38, very ->
➥19, simulates -> 19, simple -> 19, stream -> 38)
[17:50:14,505] [DEBUG]--current count HashMap(this -> 20, a -> 40, very ->
➥20, simulates -> 20, simple -> 20, stream -> 40)
```

❶ When new text is ready to be sent to a worker

❷ When text comes back and is merged over previous counts

The lag is zero because the workers are available and all texts sent at this time are processed without any problems. Let’s dive into the code.

8.4.2 The code

Part of the entry point for starting the application is shown in listing 8.7. Some of it is related to the `role` and `port` parameters specified at startup. If no parameters are passed, an aggregator is created with a random port for the `akka.remote` configuration. Otherwise, the parameters can specify the port and role of the node.

Listing 8.7 Entry point of the application